Short Questions 7

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Abstract

This paper addresses three topics spanning the transition from tabular to function approximation methods in reinforcement learning: the motivation for extending TD learning beyond a single step, the n-step return formulas for SARSA-like algorithms, and the limitations of table-based RL that motivate the use of deep learning as a function approximator.

Question 1

What are arguments for going beyond one step in a temporal difference (TD) learning by an agent that learns by interacting with an environment?

TD(0) bootstraps after a single step, replacing the rest of the return with an estimate $V(S_{t+1})$. This is fast and low-variance, but it puts a lot of trust in that one-step estimate. If V is inaccurate—especially early in learning—the update target is misleading, and the agent corrects slowly because it only sees one real reward before bootstrapping.

Going to n-step TD lets us use more actual observed rewards before plugging in an estimate. The n-step return $G_{t:t+n}$ includes n real transitions, so the target carries more ground-truth information and less dependence on a potentially wrong value function. At the extreme, $n = \infty$ recovers the Monte Carlo return with no bootstrapping at all. In between, we get a bias–variance tradeoff: larger n reduces bias (more real rewards) but increases variance (more stochastic transitions in the target). Sutton & Barto (Figure 7.2) demonstrate this empirically—neither $n = 1$ nor the full MC return performs best on the 19-state random walk; an intermediate n achieves the lowest error (Sutton and Barto 2018).

Multi-step methods also propagate reward information faster. A one-step method needs many sweeps for a distant reward to ripple back through the value function. An n-step method can carry that reward signal n states back in a single update, which is especially valuable in sparse-reward environments where the agent may go many steps between meaningful feedback.

Question 2

Write the formula for the return G_t for an n-step SARSA-like TD algorithm for the two cases: i) when it is used to estimate $v(s)$, and ii) when it is used to estimate $q(s, a)$.

Case i: Estimating $v(s)$. The n-step return for state-value estimation (Sutton & Barto, Eq. 7.1) is:

$$G_{t:t+n} = R_{t+1} + \gamma\, R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n\, V(S_{t+n}) \tag{1}$$

The first n terms are discounted real rewards observed over the next n transitions. The final term bootstraps off the current estimate $V(S_{t+n})$. The state-value update is then $V(S_t) \leftarrow V(S_t) + \alpha\big[G_{t:t+n} - V(S_t)\big]$.

Case ii: Estimating $q(s, a)$. The n-step return for action-value estimation in SARSA (Sutton & Barto, Eq. 7.7) is:

$$G_{t:t+n} = R_{t+1} + \gamma\, R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n\, Q(S_{t+n}, A_{t+n}) \tag{2}$$

The structure is identical, but instead of bootstrapping off a state value, we bootstrap off the action value $Q(S_{t+n}, A_{t+n})$, where A_{t+n} is the action actually selected at step $t + n$ under the current policy. The action-value update is $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[G_{t:t+n} - Q(S_t, A_t)\big]$.

In both cases, if the episode terminates before n steps (i.e., $t + n \geq T$), the return simply becomes the truncated sum of rewards with no bootstrap term—it reduces to the standard MC return from that point.

Question 3

Traditional reinforcement learning algorithms are table-based that store $v(s)$ and/or $q(s, a)$ values and update them. List some problems with such table-based reinforcement learning algorithms. How can attempts be made to overcome some of these problems? How does machine learning, in particular, deep learning, may be of help?

Tabular methods store one entry per state (or state-action pair), and this creates several problems as environments grow in complexity.

Curse of dimensionality. The number of states grows exponentially with the number of state variables. A grid world with k dimensions and m cells per dimension has m^k states. Real-world problems—robot joint angles, game screens, continuous physics—easily produce state spaces

too large to fit in memory, let alone visit often enough to get accurate estimates.

No generalization. Each table entry is independent. Visiting state s_{42} tells us nothing about the value of nearby state s_{43}. The agent must visit every single state (and take every action in it) to build an accurate value function. In large spaces, most states will never be visited, so the table stays full of uninitialized entries.

Continuous states and actions. Many real problems have continuous state spaces—a table cannot store an entry for every possible real-valued state. Discretization is a workaround, but fine grids explode the table size and coarse grids lose information.

Function approximation addresses these issues by replacing the table with a parameterized function $\hat{v}(s; \mathbf{w})$ or $\hat{q}(s, a; \mathbf{w})$ that maps states (or state-action pairs) to values using a weight vector $\mathbf{w}$ with far fewer parameters than the number of states. When we update the weights after visiting one state, the change affects the predicted values for similar states too—this is generalization, and it is what tables fundamentally lack (Sutton and Barto 2018).

Deep learning takes this further. A deep neural network can take raw, high-dimensional input—like an image of a game screen—and learn its own feature representation through multiple layers of nonlinear transformations. Deep Q-Networks (DQN) demonstrated this by playing Atari games directly from pixels, using a convolutional neural network as the function approximator for $Q(s, a)$ in place of a table (Mnih et al. 2015). The network learns to extract relevant features (object positions, velocities) without manual engineering. Techniques like experience replay and target networks stabilize training, which is necessary because the non-stationary, correlated nature of RL data can otherwise destabilize neural network learning.

AI Use Statement

Claude (Anthropic) was used to format my answers in AAAI LaTeX format.

References

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition. Available free at http://incompleteideas.net/book/the-book-2nd.html.